Marconi Research and Innovations Lab

Infrastructure Operations Agent

Design and Implementation

Abstract

Diagnosing a fault in a small self-hosted environment usually means opening three separate interfaces and correlating them by hand: the container runtime for process state, a metrics server for resource behaviour, and a log store for what the service itself reported. The work is mechanical, repeats every incident, and takes longest exactly when an operator can least afford the delay.

This report describes a system that performs that correlation on request. An operator asks a question in ordinary language from a chat client; a language model, given a fixed set of read-only query tools, decides which backends to consult, issues the queries, and returns an answer with the evidence it rests on, a likely cause, and recommended next steps. The design has one governing constraint: the agent can observe but cannot act. No tool in the system issues a mutating call to any backend, so the worst outcome of a mistaken model turn is an incorrect answer rather than a stopped service — a guarantee enforced by the set of operations exposed to the model, not by instructions given to it.

Beyond the conversational path, the system polls container and endpoint health on a fixed cycle, alerts on transitions, records them durably, and publishes daily and weekly reports that keep measured data separate from the generated prose summarising it. The report describes how the system is built and why, and closes with the design decisions behind it, what each one costs, and the limitations that remain.

Contents

Abstract **ii**

1 Introduction **1**

2 Architecture **3**

3 Agent state and tools **7**

4 Monitoring and reporting **9**

5 Interface and configuration **12**

6 Deployment **13**

7 Design decisions and outcome **14**

References **16**

1 Introduction

1.1 The operational problem

A small self-hosted environment has three separate sources of truth about its health: the container runtime (process state), a metrics server (time series), and a log store (what each service reported). None answers a non-trivial question alone. Diagnosing a restart needs runtime state, the logs immediately before it, and the resource curve over the preceding interval, and an experienced operator runs roughly that sequence for every incident, which branches on what each step returns and so resists a fixed script.

1.2 Approach

The model discovers the environment rather than being told about it: enumeration operations (services, metric names, label values) exist alongside query operations, and the model is instructed to enumerate before it queries, since a guessed name returns nothing.

Access runs through the existing chat workspace rather than a purpose-built interface, so a question, an alert, and a scheduled report all land in the place an operator already watches.

1.3 Scope

1. **Conversational investigation.** Free-form questions, answered with every read-only operation available, context retained per conversation.
2. **Structured incident reporting.** The same loop on a fixed template: summary, observed behaviour, evidence, likely cause, next steps.
3. **Continuous monitoring.** Container and endpoint state polled, and configured metric rules evaluated, on a fixed cycle; transitions produce alerts and durable records.
4. **Scheduled reporting.** Daily and weekly digests of current state and the transitions recorded over the window.

1.4 Out of scope

1. **No write operations.** Nothing restarts, scales, reconfigures, or deletes; the design constraint Section 3.5.5 enforces.
2. **Single tenant.** One environment, one chat workspace, no per-user authorisation beyond what the workspace provides.
3. **No model training.** The model is an external service reached over a network API.

1.5 System overview

The system runs as a single long-lived process: a chat connection, a reasoning loop that answers questions, a polling loop that watches health, and a scheduled report task. Figure 1 shows the arrangement.

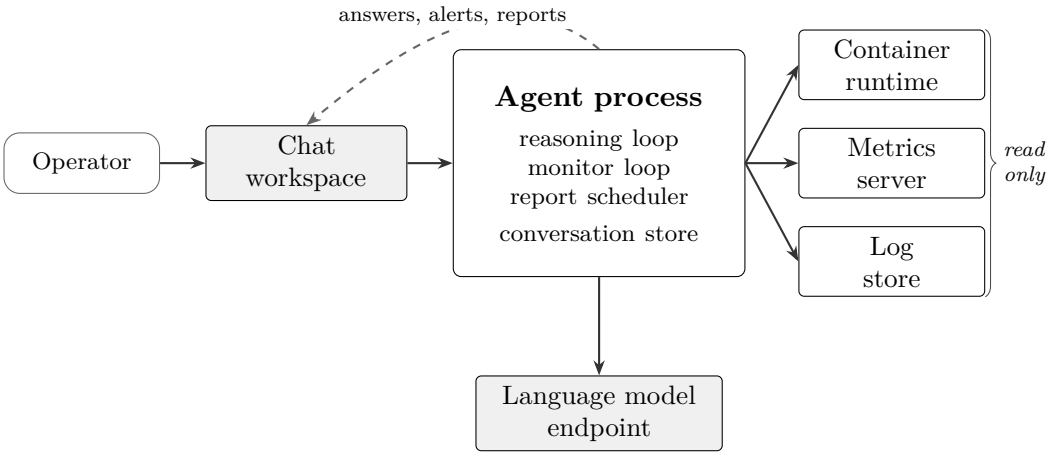


Figure 1: Top level arrangement. The agent process is the only component that talks to both the chat workspace and the observability backends. All backend access is read only.

1.5.1 The conversational path

A slash command, direct message, or mention all reach the same reasoning loop: consult the model, execute the requested operations, feed back the results, repeat until it answers without requesting more. The answer is reformatted for the platform and split across messages past the length limit.

1.5.2 The monitoring path

A polling task on a fixed interval reads container state and probes each configured endpoint, comparing against the last cycle. A change (running/not, reachable/not) is a transition: logged durably and posted as an alert, with recoveries on a separate channel. Configured metric rules follow the same pattern, and a rule that starts firing can also start an investigation.

1.5.3 The reporting path

A scheduled task fires daily, rendering current state and the day's transitions, plus a weekly roll-up of per-service outage counts and downtime on one configured weekday. Where the publish channel is a forum, each digest opens its own thread.

2 Architecture

2.1 Layering

The system is arranged in layers with imports running strictly downward (Figure 2); each is defined as much by what it refuses to know as by what it does.

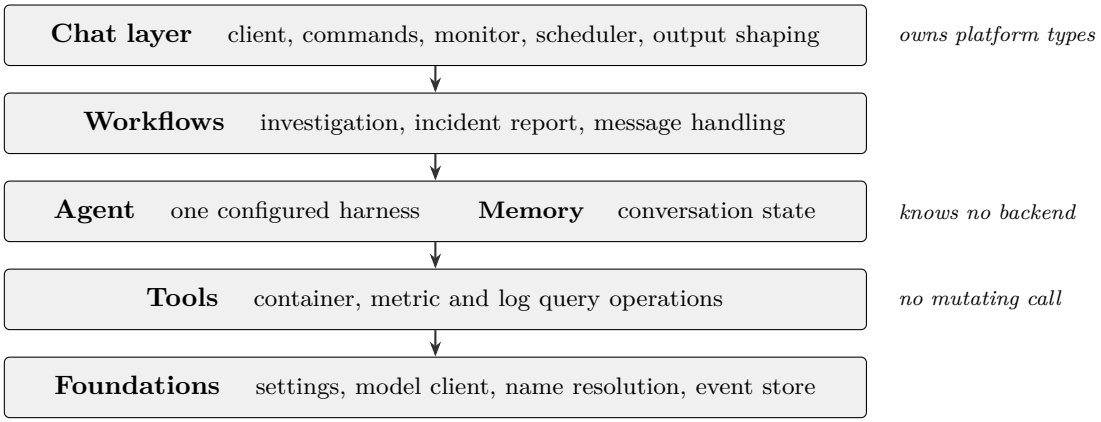


Figure 2: Layering. Imports run downward only. The agent sits below the workflows that call it and above the tools it is constructed with.

2.1.1 Foundations

Settings read configuration from the environment once at import into a validated, typed object, resolving both storage paths so every component agrees which file is which. It also derives the model identifier (a `provider:model` string) and publishes the credential and endpoint to the process environment, so any server speaking the OpenAI-compatible protocol [3] is usable, including a self-hosted model. Name resolution provides the one matching rule used for every service lookup (Section 3.6). The event store persists the transition log and the watch list.

2.1.2 Tools

Each tool module pairs a thin asynchronous HTTP client [6] for one backend with the model-callable operations built on it: the client is ordinary code other components call directly, the tool half adds the schema, the model-facing description, and argument clamping. This layer knows nothing about conversations, chat platforms, or scheduling, and contains no operation that changes anything (Section 3.5.5).

2.1.3 Agent and memory

One module constructs the agent — the only place in the system that does — naming the model, tool set, operations persona, middleware stack, and conversation store in a single call to the framework's factory. Built once and cached, every workflow shares the instance, so there is exactly one answer to what tools the model has and what instructions it runs under (cost: Section 7). Memory holds conversation state keyed by thread, as a peer of the agent rather than a part of it, so its lifetime belongs to the process rather than to any one conversation.

2.1.4 Workflows

A workflow is one unit of work. Investigation binds the full tool set to the operations persona [2]; incident reporting calls it with a fixed template. Both are plain async functions returning a string, with no chat platform types in their signature, which keeps the chat layer a thin adapter.

The one exception is message handling: it receives platform message objects because its job is deciding whether an arbitrary incoming message is addressed to the agent at all (placed here rather than in the chat client for the reason in Section 5.5).

2.1.5 Chat layer

Everything that knows the platform's types lives here: the client and its lifecycle, commands, the monitoring task, the report scheduler, output shaping, and the bridge forwarding application logs into a channel.

2.2 Request flow

Figure 3 traces one investigation from question to answer.

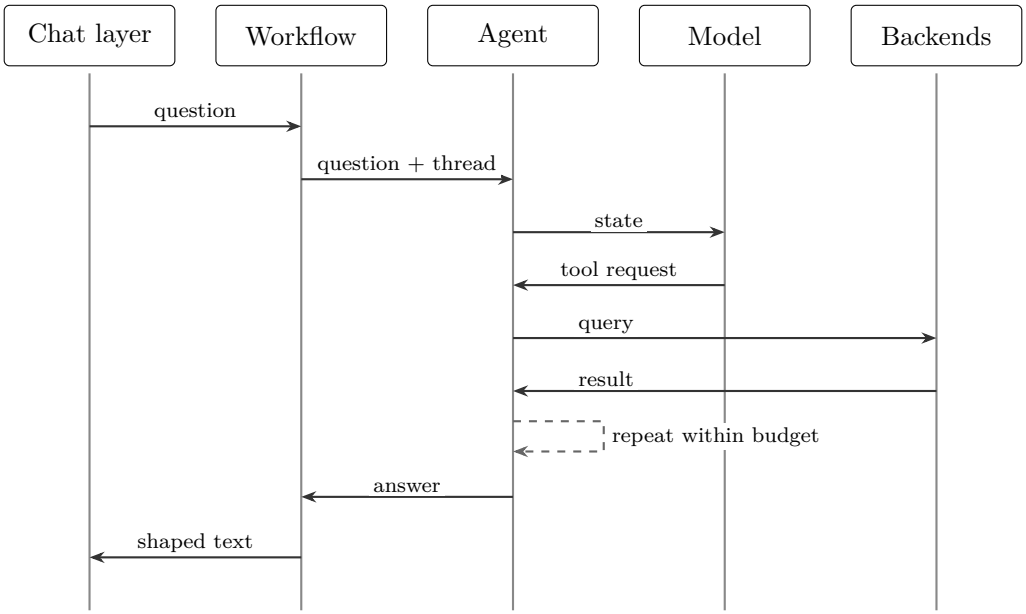


Figure 3: One investigation. The agent mediates between the model and the backends. The model never reaches a backend directly; it asks the agent to, and reads the result as text.

The model issues no network calls: it emits a structured request naming an operation and its arguments, and the harness looks that name up in the tool set the agent was built with. An operation outside that set cannot be invoked regardless of what the model emits, making the tool set an enforcement point, not a convention.

2.3 Data model

Two embedded database files sit side by side, each opened by exactly one component. The operational file holds the watch list (services explicitly monitored regardless of the general scope rule) and the transition log (one row per observed state change), written synchronously by the event store. The conversation file holds the memory component's per-thread checkpoints; nothing else opens it. The split avoids putting an asynchronous model-turn write and a synchronous polling-task write in contention for one lock (cost: Section 7). The transition log is trimmed on write, retaining ninety days, since a restart loop would otherwise grow it indefinitely.

2.4 The agent

2.4.1 The harness

The agent is one call to the framework's factory, made once and cached, naming the model client, tool set, operations persona, middleware stack, and conversation store. Everything the loop does between a question and an answer is the framework's, shown in Figure 4.

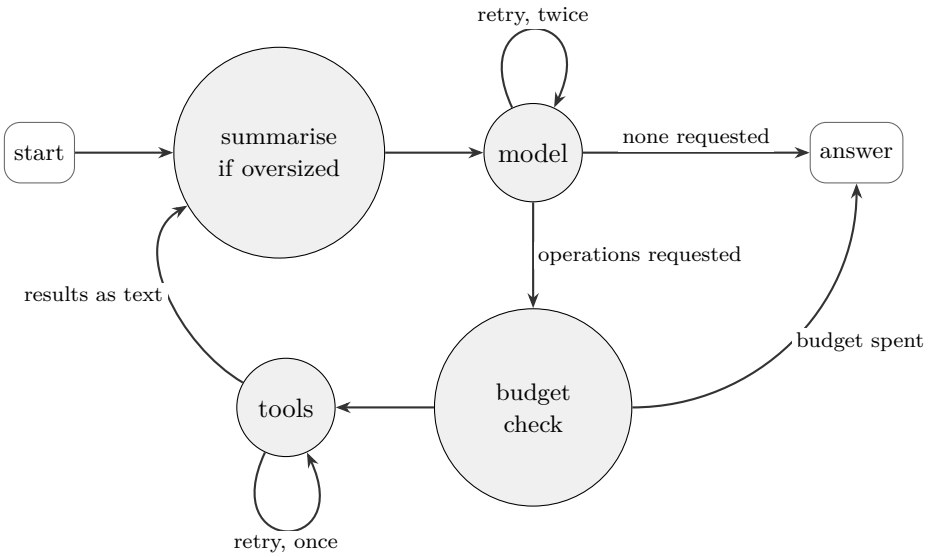


Figure 4: The agent as configured. Middleware brackets both nodes: the history is compressed before the model call and the call retried twice on failure; the tool budget is enforced after it, and an operation that raises is retried once before its failure becomes the result text on the return edge. The cycle through the tools and back is the ordinary loop. Not drawn: each model request carries a thirty-second timeout and the whole run a sixty-second one, both outside the graph.

Termination is decided by whether the model's most recent message requested any operations: if so, they run and control returns to the model; if not, that message is the answer. No separate stopping heuristic, no cap on the answer itself.

2.4.2 The middleware stack

Five middleware components, each replacing code this system previously wrote by hand:

Middleware	**Behaviour**
Model retries	Failed model call retried up to twice before the run is abandoned.
Operation retries	A raising operation retried once before its error is handed on.
Context summarisation	History over the token budget is replaced with a generated summary before the model call; may target a cheaper model than the one answering.
Tool budget	A run may spend a fixed number of operations; the call that would exceed it, and any sibling call in the same turn, is not executed.
Operation errors	A raising operation becomes an ordinary result carrying the failure text, which the model reads and continues from.

2.4.3 The tool budget

Unbounded loops against a paid or rate-limited endpoint are a liability, so the number of operations one question may consume is bounded, counting operations rather than turns (a turn requesting three operations in parallel spends three). Exceeding the budget produces no error: the run ends with the model's own account of what it had established, and that partial answer is delivered normally.

2.4.4 Failure handling

An operation that raises returns its exception text as the result; a nonexistent operation the model names is reported as such rather than as a failure. Retries bound both: a model call up to three times total, an operation twice. Two time ceilings bound the run: thirty seconds per model request, sixty seconds overall. The investigation workflow catches everything above the loop and returns the failure as its answer, so a caller always gets a string (why: Section 7).

3 Agent state and tools

3.1 Threads

Conversation state is keyed by the channel or direct message the question arrived in, so two channels never share history. Omitting a thread id gives a one-shot exchange with no persistence, incident reports use this path so a report describes the service, not whatever the channel discussed earlier. There is no interface to read, write, or delete a stored thread from outside the component that owns it.

3.2 Durability

Conversation state lives in its own database file [7], so a thread survives a restart. The storage component holds one connection, opened on first use inside its event loop and kept for the process lifetime.

3.3 Bounding a long conversation

A thread grows without bound while the model's context does not, so history past a configured token budget is compressed: older turns become a generated summary, recent turns stay verbatim.

3.4 Stated limits

Memory means one conversation replayed per thread: no recall across threads, no retrieval over past ones. Compression is also lossy by construction, a long conversation keeps only a summary of its early turns, not the turns themselves.

3.5 The tool surface

3.5.1 Composition

Ten operations are available to the model, covering three backends. Table 1 summarises them.

Operation	Purpose
`list_services`	Enumerate container names and configured endpoints
`docker_status`	State, status and image of one container
`docker_logs`	Recent output of one container
`list_metric_names`	Enumerate available metric names
`get_label_values`	Enumerate values of a metric label
`query_prometheus`	Instant query for current values
`query_prometheus_range`	Range query for behaviour over a window
`get_loki_label_values`	Enumerate values of a stream label
`query_loki`	Log query over a short recent window
`query_loki_range`	Log query over an explicit window

Table 1: The ten read-only operations available to the model.

Four of the ten exist purely for discovery, deliberately: a model asked about resource usage will otherwise query a plausible-sounding metric name that does not exist, get an empty result, and report usage as zero. Cheap enumeration, with an instruction to enumerate first, turns that confident wrong answer into a correct one.

3.5.2 Output discipline

Every operation returns text and bounds it: results truncate to a fixed length, enumerations cap at a fixed count, protecting the context window a busy log stream would otherwise consume outright. Range arguments clamp rather than reject, a twenty-four-hour log request against a six-hour maximum returns six hours, not an error.

3.5.3 Backend notes

- **Container runtime [8].** Reached over a local socket or network address; an unreachable runtime returns an empty list, not an error, so a metrics question stays answerable without it. Output framing is parsed and stripped before the text reaches the model.
- **Metrics [10, 11].** Instant queries return current values; range queries return a series at a resolution derived from the window length, so a thirty-minute and a six-hour query cost about the same to transmit and read.
- **Logs [12, 13].** The instant endpoint rejects plain stream selectors, so instant queries run as a short-window range query instead. Stream labels share no vocabulary with container names or metric labels, so the log operations tell the model to enumerate real labels before writing a selector.

3.5.4 Input validation

Label names reach a URL path segment, so they are validated against the backends' permitted character set before use; a name that fails validation returns an empty result, not a request. This is a trust boundary, model output is shaped by tool results that themselves derive from log lines an arbitrary service produced, so it is not trusted input, and path separators, query markers, and whitespace are all rejected.

3.5.5 Read only by construction

The guarantee that the agent cannot act rests on Table 1 alone: every operation is a query, none accepts a state-changing parameter, none reaches a mutating endpoint. The model is also instructed to recommend actions rather than perform them, which improves answers but is not the guarantee, instructions constrain what a model tries, the tool table constrains what it can do. Section 6.4 returns to this for runtime socket access.

3.6 Name resolution

Operators type approximate names, a project prefix here, a role suffix there, hyphens in one tool and underscores in another. One rule handles it everywhere: a name is reduced by stripping separators and lowercasing, and a candidate matches if the reduced query appears inside the reduced candidate.

1. **Exact match wins**, so a service whose name is a prefix of a longer one stays addressable.
2. **Ambiguity is an error**: several partial matches with no exact match fail with a message listing the candidates, rather than silently picking one.

The second rule is the one that matters operationally: silently choosing among matches would produce a believable status report about the wrong service. An explicit ambiguity message costs one round trip and cannot mislead. The same rule applies to watch list lookups.

4 Monitoring and reporting

4.1 The poll cycle

A single task runs on a configured interval with a floor applied, so a misconfigured interval cannot produce a tight polling loop. Each cycle reads all container states, probes each configured HTTP endpoint, and evaluates each configured metric rule.

4.2 Scope

Container monitoring is off by default. A container is in scope if its name contains a configured filter substring, or an operator added it to the watch list; an empty filter puts every container in scope, covering both a single-purpose host and one sharing unrelated workloads.

4.3 Transition detection

The task holds each target's previous observation and compares: running to not-running is a failure, the reverse a recovery, and endpoints follow the same pattern over a reachable flag (a non-error response is reachable; an error status, a connection failure, or a timeout is not). Figure 5 shows the resulting behaviour, including the case that produces no output.

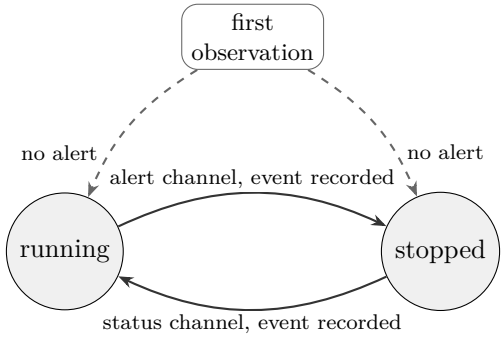


Figure 5: Transition handling. Failures and recoveries are routed to different channels. The first observation of a target establishes a baseline and produces no alert.

4.4 Baselining

The first observation of any target records its state and emits nothing, otherwise every restart would announce every container already down, training operators to mute the alert channel. The cost: a container that fails while the agent is not running is baselined silently once the agent returns, though it still shows, marked not running, in the report's current-state section.

4.5 Reading state from the runtime rather than querying metrics

Liveness could be derived from a metrics query instead, since the metrics server already tracks whether its scrape targets respond. The runtime is read directly for two reasons: a metrics-derived signal lags the scrape interval and reports the transition of a scrape rather than of a process, and if the metrics server itself restarts, every target it scrapes appears to fail at once, an alert built on metric queries would announce a total outage when the monitoring system just bounced. This decision concerns container liveness only; metric conditions can still drive alerts through explicitly configured rules.

4.6 Endpoint probing

Endpoint checks catch what runtime inspection cannot: a service on another host, behind a proxy, or a health path whose success depends on more than the process being alive.

4.7 Metric rules

Each configured rule is a named metrics query, and it fires when the query returns any series. Rules follow the same baseline-then-transition pattern as containers and endpoints. A query the metrics server rejects, or cannot answer, is skipped rather than read as resolved, since being unable to check is not evidence that the condition cleared. A rule that starts firing posts an alert and, when an incident channel is configured, starts an agent investigation of it outside the poll cycle, so a slow model cannot delay the next check.

4.8 The event log

Every transition is recorded durably before its alert is dispatched, so a channel-less environment still accumulates a full history. Live state is held only in memory and rebuilt each cycle; a target leaving scope drops its state, so a service returning later is baselined fresh rather than compared to something stale.

4.9 Scheduled reporting

4.9.1 Schedule

A single task fires once daily at a configured hour, always publishing the daily digest and, on one configured weekday, the weekly roll-up too; both are suppressed with no reports channel configured. There is no catch-up for a missed firing, a digest delivered late on restart is a distraction rather than a report.

4.9.2 Periodic status

A second loop publishes a status snapshot at a configured interval (two hours by default) to the status channel, reporting current state for every in-scope target; suppressed with no status channel configured. Unlike a transition alert the snapshot is a level, it fires whether or not anything changed, so a healthy snapshot is positive evidence the observer is alive. The first fires on startup and renders the same current-state section the daily digest opens with.

4.9.3 Contents

The daily digest gives current state plus every transition from the preceding twenty-four hours, in order. The weekly roll-up gives the same current state plus per-target totals: outage count and cumulative downtime over the window. Any target that failed twice or more in the window is handed to the agent, whose investigation is added as a section explaining why.

4.9.4 Downtime accounting

Totals pair each failure event with the recovery that follows it. An unmatched failure means the target is still down, that outage is measured to the present and flagged rather than dropped, discarding it would report a three-day outage as no downtime at all.

4.9.5 Measured data and generated prose

Digests separate two kinds of content, and the boundary is enforced. The detail section is rendered directly from recorded events and the model never rewrites it, passing an exact value

through a language model can only preserve or damage it. A summary section is generated on top of it for a short narrative before the numbers, and degrades to the raw detail if the model is unavailable.

4.9.6 Publication and follow-up

A report publishes as one unit, one thread where the platform supports it, split only for the per-message size limit.

5 Interface and configuration

5.1 Commands

Ten commands are registered through the chat library's application command tree [4, 5], listed in Table 2. Commands whose output concerns only the caller reply privately, so that routine list queries do not fill a shared channel.

Command	Argument	Effect
`status`	none	Current containers and endpoints, summarised
`investigate`	question	Free-form investigation with all operations
`report`	service	Structured incident report for one service
`digest`	period, publish flag	Render a scheduled report immediately
`containers`	none	Containers grouped by state
`watching`	none	Endpoints the monitor probes
`watch`	container	Add a container to the watch list
`unwatch`	container	Remove a container from the watch list
`watchlist`	none	Containers currently watched
`help`	none	List the commands

Table 2: The command set.

5.2 Conversational access

A direct message, or a mention in a channel the agent can see, routes through the same investigation path as the explicit command, mention markup stripped first; a mention with no question prompts for one. Because the thread id is the channel, an operator can hold an ordinary exchange, ask, then why, then for evidence, without restating the subject.

5.3 Output shaping

Model output needs two corrections before it is fit to send: the platform's restricted markdown dialect shows heading syntax literally and renders tables as pipe-character rows, so headings are rewritten to bold text and tables to labelled bullet lines (code fences are exempt); and answers over the platform's message-length cap are split, preferring line boundaries so a split does not land inside a word.

5.4 Log forwarding

Informational-and-above application logs forward to a configured channel, making the agent observable through the interface it serves. Library records are dropped, to avoid the feedback loop of a sent message logging an event that sends a message; forwarding is asynchronous and batched for one second per message; the queue is bounded at a thousand records, beyond that records drop rather than growing memory. What reaches the channel is the operational trail, transitions, alerts, reports, and the operations each question called and its duration, not debug noise. A drain failure is written to standard error rather than logged, to avoid feeding the forwarder that failed.

5.5 Failure must be visible

The chat library logs a handler exception to its own logger, exactly the logger excluded from forwarding above, so a failure answering a mention would go unseen and the agent would simply not reply. Message handling is instead wrapped in a workflow that always sends an explicit reply on failure. Section 7 states why an error message beats silence.

6 Deployment

6.1 Packaging

One container image, on a slim Python [1] base, with dependencies resolved from a lock file [14] and installed in a layer built before the application source is copied, so a source change reuses the cached dependency layer. The compose description [9] runs two services from that image: the Discord bot, and a read-only HTTP API mirroring the same status, investigation, report, and watch-list workflows, every route but `/health` guarded by an optional key header. The API binds loopback only by default; host networking would otherwise expose it on the LAN.

6.2 The observability backends

Neither service starts the metrics server or log store; both are pre-existing infrastructure addressed by configuration, and each is independently optional — with either address unset, its operations drop from the tool set. The log store's labels are set by whatever ships logs into it, not by this system, which is why label values are discovered rather than hard-coded (Section 3.5).

6.3 Networking and persistence

The agent shares the host's network namespace, reaching the backends and the model endpoint at local addresses; bridge-networked components reach it through the host gateway instead of by service name. The storage file lives on a mounted directory so the watch list, transition history, and conversation threads survive container replacement — losing that directory loses the reporting history irrecoverably.

6.4 Runtime socket access

The agent needs the container runtime's API socket to read container state and output. The socket is mounted read-only, but that flag governs only the socket file: the API reachable through it stays fully writable, and access to it is equivalent to root on the host.

6.5 Operating the system

The process restarts automatically unless explicitly stopped, and shutdown closes the conversation store before the event loop ends. The log channel and the container's own output both carry informational records and above in real time; the container output is where to look when the chat connection itself is the problem. Transport-level logging stays pinned to warnings to avoid flooding both destinations.

7 Design decisions and outcome

7.1 Limitations and extensions

7.1.1 Limitations

1. **Answer quality tracks model quality.** The system supplies evidence and structure; the reasoning is the model's, and no automated test covers it.
2. **Memory does not span conversations.** A thread is a conversation; nothing is recalled across threads.
3. **Long conversations keep a summary, not a record.** Past the context budget, early turns are replaced by generated prose — a detail mentioned once survives only if the summarising model kept it.
4. **Monitoring has a blind interval.** The first observation after a restart is a baseline, so a failure during downtime is never announced, though it remains visible in the next report.
5. **Missed report firings are skipped.** No catch-up: an agent down at the scheduled hour omits that day's digest.
6. **The deployment is single tenant.** Authorisation is whatever the chat workspace enforces; anyone who can message the agent can query every service it can see.

7.1.2 Possible extensions

1. **Persisting monitor state** would close the blind interval by comparing the first post-restart observation against the last one before it.
2. **Write operations**, if ever added, belong in a distinct tool set behind an explicit approval step, not folded into the existing surface — the value of Section 3.5.5's guarantee is in it being absolute.
3. **Additional backends** fit the existing pattern: a client, a set of query operations, an entry in the tool list. Nothing else changes.

7.2 Conclusion

The system automates a repetitive part of operations work: gathering evidence from three separate systems and correlating it into an account of what happened, through a language model given a fixed set of query operations and reached from the chat workspace an operator already uses.

Its most consequential property is what it cannot do. No operation available to the model changes anything, placing the guarantee in the composition of the tool set rather than in instructions the model may disregard — the constraint that lets an autonomous reasoning loop run against a live environment while the worst outcome stays an incorrect answer.

The remaining design follows from taking failure seriously in a system whose central component is probabilistic:

- **The reasoning loop is bounded**, and returns a partial summary instead of an error when the bound is reached.
- **Operation failures become evidence** rather than exceptions.
- **Report figures are rendered from recorded data** and never rewritten by the model, which restricts generation to the parts where fluency helps.

- **Monitoring baselines before it alerts**, because an alert stream that cries wolf on every restart is worse than none.

Every one of these choices costs something, and Section 7 states the costs alongside the reasoning.

References

[1] Python Software Foundation, "Python 3.13 documentation." [Online]. Available: https://docs.python.org/3.13/. [Accessed: Aug. 18, 2026].

[2] LangChain, Inc., "LangChain Python documentation: agents." [Online]. Available: https://docs.langchain.com/oss/python/langchain/agents. [Accessed: Aug. 25, 2026].

[3] OpenAI, "Chat Completions API reference." [Online]. Available: https://platform.openai.com/docs/api-reference/chat. [Accessed: Aug. 18, 2026].

[4] Rapptz and contributors, "discord.py documentation." [Online]. Available: https://discordpy.readthedocs.io/. [Accessed: Aug. 18, 2026].

[5] Discord, Inc., "Discord Developer Portal: API reference." [Online]. Available: https://discord.com/developers/docs/. [Accessed: Aug. 18, 2026].

[6] Encode OSS Ltd., "HTTPX: a next-generation HTTP client for Python." [Online]. Available: https://www.python-httpx.org/. [Accessed: Aug. 18, 2026].

[7] SQLite Consortium, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html. [Accessed: Aug. 18, 2026].

[8] Docker, Inc., "Docker Engine API reference." [Online]. Available: https://docs.docker.com/reference/api/engine/. [Accessed: Aug. 18, 2026].

[9] Docker, Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/. [Accessed: Aug. 18, 2026].

[10] Prometheus Authors, "Prometheus HTTP API." [Online]. Available: https://prometheus.io/docs/prometheus/latest/querying/api/. [Accessed: Aug. 18, 2026].

[11] Prometheus Authors, "Querying Prometheus: PromQL basics." [Online]. Available: https://prometheus.io/docs/prometheus/latest/querying/basics/. [Accessed: Aug. 18, 2026].

[12] Grafana Labs, "Grafana Loki HTTP API." [Online]. Available: https://grafana.com/docs/loki/latest/reference/loki-http-api/. [Accessed: Aug. 18, 2026].

[13] Grafana Labs, "LogQL: Loki query language." [Online]. Available: https://grafana.com/docs/loki/latest/query/. [Accessed: Aug. 18, 2026].

[14] Astral Software, Inc., "uv: Python packaging documentation." [Online]. Available: https://docs.astral.sh/uv/. [Accessed: Aug. 18, 2026].